

INTRODUCTION

1. Introduction to E-Commerce Management System

In today's digital world, online shopping has become an essential part of daily life. Businesses are increasingly shifting from traditional physical stores to online platforms to reach a wider audience. An **E-Commerce Management System** is a software application that helps manage products, inventory, pricing, and other operations of an online store efficiently.

This project titled "**E-Commerce Management System using Flask and MongoDB**" is a web-based application developed to manage products in an online store. The system allows users (admin) to:

- Add new products
- View all available products
- Edit existing product details
- Delete products from the database

This type of system is very useful for small and medium businesses to maintain their product records digitally instead of using manual registers.

1.1 Purpose of the Project

The main purpose of this project is to:

- Understand web application development using **Python Flask**
 - Learn database connectivity using **MongoDB**
 - Implement CRUD operations (Create, Read, Update, Delete)
 - Develop a simple but functional e-commerce backend system
-

1.2 Technologies Used

This project uses the following technologies:

- **Python** – Programming language
- **Flask** – Web framework

- **MongoDB** – NoSQL database
- **HTML** – Frontend structure
- **Jinja2** – Template engine for dynamic content
- **PyMongo** – MongoDB driver for Python

These technologies together form a complete full-stack mini project.

1.3 Scope of the Project

The scope of this project includes:

- Managing product details (name, price, stock)
- Performing CRUD operations
- Displaying product list dynamically
- Maintaining product data in MongoDB database

However, this project does not include:

- Payment gateway integration
- User authentication system
- Order management
- Cart functionality

It is designed as a **Product Management System** for learning and academic purposes.

LITERATURE REVIEW / BACKGROUND STUDY

2. Literature Review

An E-Commerce Management System is a software platform that enables businesses to conduct commercial transactions electronically over the internet. Over the past decade, e-commerce platforms have evolved rapidly due to increased internet penetration, smartphone usage, and digital payment systems.

Many large-scale platforms such as:

Amazon

Flipkart

Shopify

have demonstrated the importance of structured product management systems. These platforms use powerful backend systems to manage millions of products efficiently.

2.1 Traditional vs Digital Product Management

Traditional Method

In traditional businesses:

Product details are stored in notebooks or Excel sheets.

Manual calculations are used for stock updates.

Errors are common due to human mistakes.

Data retrieval is slow.

Digital Method

In digital systems:

Product data is stored in databases.

Automatic updates reduce errors.

Searching and editing is fast.

Data is more secure and organized.

This project demonstrates a simplified digital product management system using modern web technologies.

2.2 Web Application Development

Web applications are software programs that run on a web server and are accessed through a browser. They are developed using:

Backend technologies (Python, Node.js, PHP)

Frontend technologies (HTML, CSS, JavaScript)

Databases (SQL or NoSQL)

In this project:

Backend: Python with Flask

Database: MongoDB

Frontend: HTML with Jinja2 templating

2.3 Introduction to Flask Framework

Flask is a lightweight web framework written in Python. It is widely used for small to medium-scale web applications.

Advantages of Flask:

Simple and easy to learn

Lightweight and flexible

Good for academic projects

Supports template rendering using Jinja2

Flask follows the concept of routing where different URLs are mapped to Python functions.

Example:

/ → Home page

/add → Add product page

/edit/<id> → Edit product page

2.4 Introduction to MongoDB

MongoDB is a NoSQL database that stores data in JSON-like documents.

Features:

Flexible schema

High performance

Easy integration with Python using PyMongo

Stores data as collections and documents

In this project:

Database Name: ecommerce

Collection Name: products

Fields: name, price, stock

2.5 CRUD Operations

CRUD stands for:

C – Create (Add Product)

R – Read (View Product)

U – Update (Edit Product)

D – Delete (Remove Product)

This project successfully implements all four operations using Flask and MongoDB.

SYSTEM OVERVIEW AND WORKING PRINCIPLE

3. System Overview

The **E-Commerce Management System** developed in this project is a web-based application that allows an administrator to manage product information efficiently. The system is built using:

- Backend: Python with Flask
- Database: MongoDB
- Frontend: HTML with Jinja2 templating

The system follows a simple client–server architecture where:

- The **client** (web browser) sends requests.
 - The **Flask server** processes requests.
 - The **MongoDB database** stores and retrieves product data.
 - The server sends responses back to the browser.
-

3.1 System Architecture

The architecture of the system consists of three main layers:

1 Presentation Layer (Frontend)

- HTML pages: `index.html`, `add.html`, `edit.html`
- Displays product data
- Takes input from the user

2 Application Layer (Backend)

- `app.py` file
- Contains Flask routes
- Handles business logic
- Connects frontend to database

3 Database Layer

- MongoDB database
- Stores product documents
- Performs insert, update, delete, and fetch operations

3.2 Working Principle

The working of the system can be explained step-by-step:

Step 1: User Accesses Home Page

When the user opens:

`http://127.0.0.1:5000/`

The Flask route `/` is executed.

```
@app.route('/')
def index():
    products = collection.find()
    return render_template('index.html', products=products)
```

- The application fetches all product records from MongoDB.
- The records are passed to `index.html`.
- The product list is displayed in table format.

Step 2: Adding a Product

When the user clicks **Add Product**:

- The `/add` route is opened.
- User enters name, price, and stock.
- On submit:
 - Data is collected from form.
 - Inserted into MongoDB using `insert_one()`.
 - Redirected back to home page.

Step 3: Editing a Product

When the user clicks **Edit**:

- The product ID is passed in the URL.
- Flask fetches product using `ObjectId`.
- Updated values are saved using `update_one()`.

Step 4: Deleting a Product

When the user clicks **Delete**:

- The product ID is passed.
- Flask executes:

```
collection.delete_one({"_id": ObjectId(id)})
```

- Product is removed from database.
 - Page redirects to home page.
-

3.3 Data Flow Diagram (DFD – Level 0 Explanation)

The system works as:

User → Flask Application → MongoDB Database → Flask Application → User

This shows how data flows between user interface and database through the backend.

DETAILED SYSTEM ARCHITECTURE

4. Detailed System Architecture Diagram

The E-Commerce Management System follows a **3-Tier Architecture Model**, which divides the system into three logical layers:

1. Presentation Layer (Frontend)
2. Application Layer (Backend)
3. Database Layer

This separation improves modularity, maintainability, and scalability of the system.

4.1 Architectural Components

1 Presentation Layer (Client Side)

This layer consists of HTML templates:

- `index.html`
- `add.html`
- `edit.html`

Responsibilities:

- Display product information
- Accept user input
- Send HTTP requests to Flask server
- Render dynamic data using Jinja2 templating engine

When the user performs actions (Add/Edit/Delete), the request is sent to the Flask backend.

2 Application Layer (Flask Backend)

The backend is implemented using Python and Flask framework.

Main responsibilities:

- Routing URLs to functions
- Handling HTTP GET and POST requests
- Processing form data
- Connecting to MongoDB
- Performing CRUD operations
- Returning rendered HTML pages

Key components in `app.py`:

- `Flask()` object creation
 - MongoDB connection using `MongoClient`
 - Route decorators (`@app.route`)
 - CRUD methods:
 - `insert_one()`
 - `find()`
 - `update_one()`
 - `delete_one()`
-

3 Database Layer (MongoDB)

The system uses MongoDB, a NoSQL database.

Database Details:

- Database Name: **ecommerce**
- Collection Name: **products**

Each product is stored as a document like:

```
{ "_id": ObjectId, "name": "Product Name", "price": 1000.0, "stock": 50 }
```

MongoDB stores data in JSON-like format which makes it flexible and easy to manage.

4.2 Architectural Flow Explanation

The complete architecture works as follows:

1. User enters data in browser.
2. Browser sends HTTP request to Flask server.
3. Flask processes request and performs required database operation.
4. MongoDB returns result.
5. Flask renders updated HTML template.
6. Updated page is displayed in browser.

This process happens dynamically without reloading the entire application server.

4.3 Advantages of This Architecture

- Clear separation of concerns
 - Easy debugging
 - Easy to upgrade database or frontend independently
 - Suitable for small to medium web applications
 - Beginner-friendly for academic projects
-

DATABASE DESIGN AND DATA MODEL

5. Database Design

The E-Commerce Management System uses **MongoDB**, a NoSQL database, to store product information. Unlike relational databases that use tables and rows, MongoDB stores data in the form of **collections and documents**.

This project uses a simple and efficient database design to manage product details.

5.1 Database Structure

The database structure used in this project is:

- **Database Name:** `ecommerce`
- **Collection Name:** `products`

In MongoDB:

- A **Database** contains collections.
 - A **Collection** contains documents.
 - A **Document** is similar to a record (row) in SQL.
-

5.2 Product Data Model

Each product is stored as a document in JSON-like format:

```
{
  "_id": ObjectId("65f1a2b3c4d5e6f789012345"),
  "name": "Laptop",
  "price": 50000.0,
  "stock": 10
}
```

Fields Description:

Field Name	Data Type	Description
<code>_id</code>	ObjectId	Unique identifier (automatically generated)
<code>name</code>	String	Name of the product
<code>price</code>	Float	Price of the product

stock

Integer

Available quantity in
inventory

5.3 Why MongoDB is Used?

MongoDB is chosen for this project because:

- It is easy to integrate with Python using PyMongo.
 - It supports flexible schema.
 - It stores data in JSON-like format.
 - It is suitable for web applications.
 - It provides fast read and write operations.
-

5.4 CRUD Operations in Database

1 Create (Insert)

```
collection.insert_one({  
    "name": name,  
    "price": price,  
    "stock": stock  
})
```

This inserts a new product document into the **products** collection.

2 Read (Fetch)

```
products = collection.find()
```

This retrieves all product documents from the database.

3 Update

```
collection.update_one(  
    {"_id": ObjectId(id)},  
    {"$set": {  
        "name": name,  
        "price": price,  
        "stock": stock  
    }}
```

```
)  
    }  
}
```

This updates specific product details.

4 Delete

```
collection.delete_one({"_id": ObjectId(id)})
```

This removes a product from the collection.

5.5 Comparison: SQL vs MongoDB

Feature	SQL	MongoDB
Structure	Tables	Collections
Data Format	Rows & Columns	JSON Documents
Schema	Fixed	Flexible
Scalability	Vertical	Horizontal

For small academic projects like this, MongoDB provides simplicity and flexibility.

IMPLEMENTATION DETAILS – BACKEND (app.py EXPLANATION)

6. Backend Implementation Using Flask

The backend of the E-Commerce Management System is implemented using **Python Flask Framework**. The main backend file is:

- `app.py`

This file controls:

- Routing
 - Form handling
 - Database operations
 - Page redirection
 - Application execution
-

6.1 Importing Required Libraries

```
from flask import Flask, render_template, request, redirect, url_for
from pymongo import MongoClient
from bson.objectid import ObjectId
```

Explanation:

- **Flask** → Creates the web application
 - **render_template** → Renders HTML pages
 - **request** → Handles form data
 - **redirect** → Redirects to another route
 - **url_for** → Generates URL for routes
 - **MongoClient** → Connects to MongoDB
 - **ObjectId** → Converts string ID into MongoDB ObjectId
-

6.2 Creating Flask Application

```
app = Flask(__name__)
```

This initializes the Flask application.

6.3 Connecting to MongoDB

```
client = MongoClient("mongodb://localhost:27017/")
db = client["ecommerce"]
collection = db["products"]
```

Explanation:

- Connects to local MongoDB server
- Creates/Accesses `ecommerce` database
- Accesses `products` collection

If the database does not exist, MongoDB automatically creates it when data is inserted.

6.4 Home Route – Display Products

```
@app.route('/')
def index():
    products = collection.find()
    return render_template('index.html', products=products)
```

Functionality:

- Fetches all product documents
- Sends data to `index.html`
- Displays product list

This implements the **Read (R)** operation of CRUD.

6.5 Add Product Route

```
@app.route('/add', methods=['GET', 'POST'])
def add():
```

Functionality:

- GET → Displays add form
- POST → Collects form data
- Converts price to float
- Converts stock to integer
- Inserts product into MongoDB

Implements **Create (C)** operation.

6.6 Edit Product Route

```
@app.route('/edit/<id>', methods=['GET', 'POST'])
```

Functionality:

- Receives product ID from URL
- Converts ID to ObjectId
- Updates product using `update_one()`

Implements **Update (U)** operation.

6.7 Delete Product Route

```
@app.route('/delete/<id>')
def delete(id):
    collection.delete_one({"_id": ObjectId(id)})
    return redirect(url_for('index'))
```

Functionality:

- Deletes selected product
- Redirects back to home page

Implements **Delete (D)** operation.

6.8 Running the Application

```
if __name__ == '__main__':
    app.run(debug=True)
```

- Runs Flask development server
 - `debug=True` enables error messages and auto-reload
-

6.9 Summary of Backend Logic

The backend successfully:

- Connects to MongoDB
- Implements CRUD operations
- Handles HTTP requests
- Passes dynamic data to templates
- Maintains product records efficiently

This makes the application fully functional.

FRONTEND & CRUD OPERATIONS

7. Frontend Implementation (HTML Templates)

The frontend of the E-Commerce Management System is developed using **HTML** and the **Jinja2 templating engine** integrated with Flask. It handles user interaction and displays dynamic data fetched from MongoDB.

Templates Used

- `index.html` – Displays product list
- `add.html` – Adds new product
- `edit.html` – Updates product

Flask automatically detects files inside the **templates** folder.

7.1 index.html – Display Products

- Shows ID, Name, Price, Stock
- Provides Edit & Delete buttons
- Contains Add Product link

Dynamic rendering using:

```
{% for product in products %}
```

Data is displayed in a structured table format.

7.2 add.html – Create Product

Form fields:

- Product Name (Text)
- Price (Number)
- Stock (Number)

```
<form method="POST">
```

Flow: User → Form Submit → `/add` route → `request.form` → `insert_one()` → Redirect to Home

Implements **CREATE** operation.

7.3 edit.html – Update Product

Existing values are pre-filled:

```
value="{{ product[1] }}"
```

Flow: User edits → `/edit/<id>` → `update_one()` → Redirect

Implements **UPDATE** operation.

Navigation Flow

Home → Add → Redirect

Home → Edit → Redirect

Home → Delete → Redirect

Advantages of Jinja2

- Dynamic content rendering
 - Supports loops & conditions
 - Reduces repetitive code
 - Easy Flask integration
-

8. CRUD Operations – Detailed Explanation

CRUD = Create, Read, Update, Delete
These are core database operations.

8.1 CREATE

```
collection.insert_one({  
    "name": name,  
    "price": price,  
    "stock": stock  
})
```

Adds new document to collection.

8.2 READ

```
products = collection.find()
```

Fetches all product documents and displays in browser.

8.3 UPDATE

```
collection.update_one(  
    {"_id": ObjectId(id)},  
    {"$set": {  
        "name": name,  
        "price": price,  
        "stock": stock  
    }}  
)
```

Updates selected product.

8.4 DELETE

```
collection.delete_one({"_id": ObjectId(id)})
```

Removes selected product permanently.

CRUD Flow Summary

User Action → Flask Route → MongoDB Operation → Updated Data → HTML Render → Display

This ensures real-time data updates.

VALIDATION, SECURITY & SYSTEM ANALYSIS

9. Input Validation and Error Handling

Input validation ensures that incorrect or harmful data is not stored in the database. In this system, validation is implemented at both frontend and backend levels.

9.1 Frontend Validation (HTML)

Example:

```
<input type="text" name="name" required>
```

```
<input type="number" step="0.01" name="price" required>
```

```
<input type="number" name="stock" required>
```

Features Used:

- `required` → Prevents empty submission
- `type="number"` → Restricts non-numeric input
- `step="0.01"` → Allows decimal values

These prevent invalid form submission at browser level.

9.2 Backend Validation (Flask)

```
price = float(request.form['price'])
```

```
stock = int(request.form['stock'])
```

Ensures:

- Price is decimal
- Stock is integer

Invalid input can be handled using `try-except` blocks in advanced implementations.

9.3 MongoDB Data Integrity

- Automatically assigns unique `_id`
 - Prevents duplicate primary keys
 - Ensures each product has unique identity
-

Common Errors & Solutions

Error	Cause	Solution
DB Connection Error	MongoDB not running	Start MongoDB service
Invalid ID	Wrong ObjectId	Pass valid ID
Empty Form	Missing input	Use required fields
Data Type Error	Wrong format	Use correct input types

10. Testing and Execution

The system was tested locally using:

- Python 3.x
- Flask
- MongoDB
- Chrome Browser

Test Cases Performed:

- ✓ Add Product
- ✓ View Products
- ✓ Edit Product
- ✓ Delete Product
- ✓ Database Update Verification

All CRUD operations executed successfully.

11. Security Considerations

The current system is suitable for academic use but lacks advanced security.

Existing Safety:

- Unique `_id`
 - Controlled routing
 - Basic data type validation
-

Security Risks:

- ❶ No User Authentication
 - ❷ No Role-Based Access
 - ❸ URL ID Manipulation Possible
 - ❹ No CSRF Protection
-

Future Security Improvements:

- Implement Login System (Flask-Login)
- Password Hashing
- Role-Based Access Control

- CSRF Tokens (Flask-WTF)
 - HTTPS Encryption
 - Secure MongoDB Configuration
-

12. Advantages and Disadvantages

Advantages:

- ✓ Simple & User-Friendly
 - ✓ Lightweight Flask Framework
 - ✓ Fast MongoDB Operations
 - ✓ Full CRUD Implementation
 - ✓ Scalable Foundation
-

Disadvantages:

- ✗ No Authentication
 - ✗ No Advanced UI Design
 - ✗ Limited Error Handling
 - ✗ No Security Protection
 - ✗ Limited Functionality (Product only)
-

FUTURE ENHANCEMENTS, REFERENCES & CONCLUSION

13. Future Enhancements

The current system is a basic academic mini-project. However, it can be upgraded into a full-scale commercial e-commerce platform by adding advanced features.

13.1 User Authentication System

- Admin Login
- User Registration
- Password Hashing
- Session Management

Tools: Flask-Login, Flask-WTF

This will restrict unauthorized access.

13.2 Role-Based Access Control

- Admin → Full Access
- Staff → Limited Access
- Customer → View Only

Improves system control and security.

13.3 Shopping Cart & Order Management

- Add to Cart
- Checkout System
- Order Placement
- Order Tracking
- Order History

Makes system similar to large platforms like Amazon and Flipkart.

13.4 Payment Gateway Integration

- UPI
- Credit/Debit Cards
- Net Banking

Possible integrations:

- Razorpay
 - Stripe
 - PayPal
-

13.5 UI & Deployment Improvements

- CSS Styling
 - Bootstrap
 - Responsive Design
 - Dashboard View
 - Cloud Deployment (Heroku, AWS, Render)
-

13.6 Additional Enhancements

- Product Image Upload
- Search & Filter System
- HTTPS Encryption
- Activity Logging
- Backup & Recovery

With these improvements, the system can evolve into a real-world commercial solution.

Official Documentation:

- [Flask Documentation](#)
 - [MongoDB Documentation](#)
 - [PyMongo Documentation](#)
 - [Python Official Documentation](#)
-

Tools Used:

- Python 3.x
 - Flask Framework
 - MongoDB Community Server
 - MongoDB Compass
 - Visual Studio Code
 - Google Chrome
-

15. Conclusion

The E-Commerce Management System using Flask and MongoDB successfully demonstrates a dynamic web-based application capable of performing complete CRUD operations.

Objectives Achieved:

- ✓ Web development using Flask
- ✓ MongoDB integration using PyMongo
- ✓ Full CRUD implementation
- ✓ Dynamic rendering using Jinja2
- ✓ Basic validation and testing

The system provides a digital method for managing product records efficiently.

Although limited to product management, it forms a strong foundation for developing a complete e-commerce platform with authentication, payment systems, and enhanced security.

From an academic perspective, this project strengthens understanding of:

- Full-stack web development
- Backend programming
- Database management
- Client-server architecture
- Software testing

Overall, the project demonstrates practical implementation of theoretical concepts studied in the B.Tech curriculum.
